

Database Indexing — Practical System Design Notes

Hinglish • SQL examples • Interview-focused • From basics to production thinking

1. Index Kya Hai?

Database index ek additional data structure hai jo database ko matching rows ko efficiently locate karne mein help karta hai. Simple analogy: book ke index se topic ka page directly locate karna.

```
Without index:
SELECT * FROM users
WHERE email = 'abc@gmail.com';
```

DB → many rows scan → matching row

```
With index:
email index → locate matching entry → row
```

Important: Index query ko automatically fast nahi banata. Query pattern aur index design compatible hona chahiye.

2. Full Table Scan vs Index Lookup

10 lakh rows

```
Full scan:
1 → 2 → 3 → 4 → ... → 1,000,000
                ↓
            matching row
```

```
Index:
query value
    ↓
index lookup
    ↓
matching row(s)
```

Agar query highly selective hai aur suitable index available hai, database ko unnecessary rows inspect karne ka work reduce ho sakta hai.

3. Real-Life Analogy

Phone book mein naam alphabetically arranged ho toh 'Sonu' locate karna easier hai. Database indexes bhi data ko organize/search karne ke liye extra structure maintain karte hain.

Lekin index ke liye extra space aur maintenance cost hoti hai—isi liye har column par index banana good practice nahi hai.

4. Common Index Types

Type / Concept	Simple use
Single-column index	One column, e.g. email
Composite index	Multiple columns, e.g. (customer_id, created_at)
Unique index	Duplicate values prevent/limit karta hai, depending on DB semantics
Primary key index	Primary key access ko support karta hai; exact implementation DB engine par depend karta hai
Full-text index	Text search use cases; normal equality index ka replacement nahi

5. Single Column Index

```
CREATE INDEX idx_users_email  
ON users(email);
```

Query:
SELECT * FROM users
WHERE email = 'abc@gmail.com';

Agar email lookup common hai aur email suitable index candidate hai, ye index query ko efficiently execute karne mein help kar sakta hai.

6. Composite Index

Jab queries commonly multiple columns ke saath filter/sort karti hain, composite index useful ho sakta hai.

```
CREATE INDEX idx_orders_customer_date  
ON orders(customer_id, created_at);
```

Query:
SELECT *
FROM orders
WHERE customer_id = 101
AND created_at >= '2026-01-01';

Key idea: Composite index mein column order important hota hai. Index ko query patterns ke according design karo.

7. Leftmost / Prefix Idea

Index:
(customer_id, created_at)

Usually useful for patterns such as:
WHERE customer_id = ?

and
WHERE customer_id = ?
AND created_at = / > ?

But a query only on created_at may not get the same benefit from this exact index, depending on the database optimizer.

Isliye composite index ka order interview mein explain karna important hai: common filtering/routing column ko appropriate position par rakho.

8. Selectivity / Cardinality

High-cardinality column mein bahut saare distinct values hote hain—jaise email, user_id. Low-cardinality example: gender/status ke limited values ho sakte hain.

High selectivity ka matlab query relatively few rows match karti hai. Such predicates often make indexes more useful, although actual optimizer decision database statistics and query shape par depend karta hai.

10 lakh users

email:
~10 lakh distinct values
→ high cardinality

status:
ACTIVE / INACTIVE
→ low cardinality

9. Unique Index

```
CREATE UNIQUE INDEX idx_users_email  
ON users(email);
```

Unique index duplicate values ko reject karne mein help karta hai according to database rules. Typical use: unique email, username, external reference number.

Note: uniqueness constraint aur index behavior database engine ke implementation details par depend kar sakte hain.

10. Index ka Cost

Index benefits:

READ → faster potential lookup

Index costs:

INSERT → index update

UPDATE → index update if indexed value changes

DELETE → index maintenance

STORAGE → extra disk/memory structures

Golden rule: 'Index everything' wrong approach hai. Important production queries identify karo aur evidence ke basis par indexes design karo.

11. Why Too Many Indexes Are Bad?

Suppose table par 15 indexes hain. Har insert/update ko relevant indexes maintain karne pad sakte hain. Large write-heavy systems mein ye overhead significant ho sakta hai.

INSERT

↓

table row

■→ index 1 update

■→ index 2 update

■→ index 3 update

■→ ...

Unused or redundant indexes periodically identify/remove karna useful ho sakta hai, but production mein removal se pehle workload analysis zaroor karo.

12. EXPLAIN / Query Plan

EXPLAIN ya database-specific execution-plan tools se pata chal sakta hai ki optimizer query ko kaise execute karne ka plan bana raha hai.

EXPLAIN

SELECT *

FROM users

WHERE email = 'abc@gmail.com';

Inspect:

→ access path

→ index used?

→ rows examined

→ join strategy

→ sort/filter work

Interview mein sirf 'index laga denge' mat bolo. Better answer: 'Query ko EXPLAIN se inspect karunga, current plan/rows examined dekhunga, phir suitable index test karunga.'

13. Query Index Use Kyun Nahi Karegi?

1. Query predicate index ke saath compatible nahi ho sakta.
2. Matching rows ka percentage bahut high ho sakta hai, so scan cheaper ho sakta hai.
3. Function/expression column par apply ho rahi ho sakti hai, depending on DB/index design.
4. Wrong column order in composite index.
5. Statistics stale/inaccurate ho sakti hain.
6. Query result ke liye table access still required ho sakta hai.
7. Optimizer ka cost model kisi aur plan ko cheaper samajh sakta hai.

14. Covering Index — Concept

Agar query ko required columns index structure se hi mil jaate hain, database ko base table se additional row lookup ki zarurat kam/avoid ho sakti hai, depending on engine. Is concept ko covering index kehte hain.

Query needs:
customer_id, created_at

Index contains enough information

↓

less table access

↓

potentially lower I/O

Exact implementation (INCLUDE columns, clustered storage, etc.) database engine par depend karti hai.

15. Index + ORDER BY

Indexes filtering ke alawa sorting ko bhi help kar sakte hain if query pattern and index ordering align karein.

```
CREATE INDEX idx_orders_customer_date  
ON orders(customer_id, created_at);
```

Potential query:
WHERE customer_id = 10
ORDER BY created_at DESC

Actual benefit query plan se verify karo. Database may still choose a sort if it considers that cheaper.

16. Index + LIKE

Potentially index-friendly:
WHERE name LIKE 'Son%'

Potentially difficult for a normal B-tree index:
WHERE name LIKE '%Son%'

Leading wildcard searches often prevent efficient use of a conventional B-tree index. Text-search requirements ke liye full-text/trigram/search-engine style solutions appropriate ho sakte hain depending on DB and use case.

17. Index + NULL / Functions

Queries involving NULL, expressions, casts, or functions can affect index usage. Modern databases support different forms of functional/expression indexes, but syntax and behavior vary.

Potential issue:
WHERE LOWER(email) = 'sonu@gmail.com'

An index on raw email may not be enough
for every database/query plan.

Possible solution:
normalized data or expression/functional index
(depending on DB)

18. Primary Key and Index

Primary key uniquely identifies rows. Most relational database systems provide an index/access structure associated with the primary key, but exact physical implementation differs (e.g. clustered vs non-clustered concepts).

Interview mein safe statement: 'Primary key uniqueness enforce karta hai and DB generally maintains an efficient access structure for it; exact storage behavior depends on the database engine.'

19. Indexing in Production — Debugging Flow

Slow API
↓
Measure p95/p99
↓
Find slow SQL
↓
EXPLAIN / execution plan
↓
Rows examined?
↓
Index suitable?
↓
Create/test index
↓
Compare before vs after
↓
Check write overhead + regression
↓
Deploy safely

Production mein index add karna bhi change management hai. Large tables par index creation ka operational impact database technology aur configuration par depend karta hai.

20. Practical Example — E-commerce

orders table:
id
customer_id
status
created_at
total_amount

Common queries:
1. customer ke recent orders
2. pending orders
3. order by id
4. date-based reports

Possible indexes should be derived from actual query patterns, e.g.
(customer_id, created_at)
or another index for a frequent status/date query.

Do not blindly create all combinations.

Schema + workload dono dekhkar indexes choose karo. Query frequency, selectivity, write volume and storage cost matter.

21. Indexing Interview Q&A;

Question	Short answer
Index kya hai?	Extra data structure that can help database locate/query rows more efficiently.
Index kyun use karte hain?	Read/query work reduce karke latency/IO improve karne ke liye.
Har column par index?	No; indexes have storage and write-maintenance costs.
Composite index?	Multiple columns ka combined index.
Column order important?	Yes, query patterns and index ordering determine usability.
High cardinality?	Many distinct values; often useful for selective lookups.
EXPLAIN?	Query execution plan inspect karne ka tool/statement.
Index use kyun nahi hua?	Optimizer cost, query shape, selectivity, statistics, expressions, or incompatible index order among possibilities.
Covering index?	Index contains enough needed data to reduce/avoid base-table lookup for a query.

Index write ko slow kyun kar sakta hai?	Insert/update/delete ke time index structures maintain karne padte hain.
LIKE '%abc%'?	Normal B-tree index often cannot efficiently support leading-wildcard search; use appropriate text-search
Primary key index?	Typically an efficient access structure is maintained, exact physical implementation depends on DB engine
Index = cache?	No. Index is database-managed access structure; cache is a separate performance layer.

22. Practice Questions

1. 10 lakh users hain aur email lookup slow hai. Tum kya check karoge?
2. Index add karne ke baad SELECT fast hua but INSERT slow ho gaya. Why?
3. Index (customer_id, created_at) hai. Kaunse query patterns likely benefit karenge?
4. Query sirf created_at par filter karti hai. Kya above composite index automatically ideal hai?
5. status column mein sirf ACTIVE/INACTIVE values hain. Is index ka benefit kab limited ho sakta hai?
6. EXPLAIN mein table scan dikh raha hai. Tum next kya investigate karoge?
7. LIKE 'Son%' vs LIKE '%Son%': difference kya hai?
8. Composite index mein column order important kyun hai?
9. Too many indexes production mein kya problem create kar sakte hain?
10. High-traffic e-commerce orders table ke indexes kaise decide karoge?
11. Read query fast karne ke liye index aur Redis cache mein difference kya hai?
12. Index add karne ke baad production performance improve hui ya nahi, kaise measure karoge?

23. Final Cheat Sheet

INDEXING CHEAT SHEET

Index:

- faster potential lookup
- extra storage
- extra write/update/delete maintenance

First investigate:

- slow query
- EXPLAIN / query plan
- rows examined
- selectivity/cardinality
- existing indexes
- query pattern

Composite index:

- multiple columns
- column order matters
- design for real queries

Production:

- don't blindly add indexes
- test impact
- watch read + write performance
- monitor storage
- remove only after evidence

Golden interview answer:

"Main pehle slow query aur execution plan check karunga. Phir query pattern ke according suitable index design karke before/after latency aur write overhead measure karunga."

24. Local Practice

Ek users/orders table banao, large dummy dataset insert karo, same query ko index ke bina aur index ke saath run karo, EXPLAIN compare karo, aur query latency measure karo. Phir composite index try karo aur INSERT performance observe karo. Ye exercise indexing ko theory se production intuition mein convert karegi.